

Software Assignment-2

Name	Entry No
Ahilaan Saxena	2023CS10076
Popat Nihal Alkesh	2023CS10058

Design Decisions:

1) Greedy Algorithm:

Motivation: As done in SW-1, we worked on devising a greedy strategy for gate placement, this time keeping in mind the sum of wire lengths while placing a particular gate onto the pre-existing 'structure':

- Observed that the gate connections form a graph like-structure. We make clusters of gates by finding disconnected components of this structure using breadth first search. This increases our efficiency as each cluster is processed independently, thus dividing our task into smaller subproblems.
- We start placing the gates in a 'layer-by-layer' kind of fashion, sorting them in decreasing order of number of connections before-hand (more the number of connections for a gate, more optimal is the layer expansion). If two gates have same number of connections, we place the gate having larger area first.
- For each cluster, we start from the first gate and place the gates directly connected to it, thus optimizing the wire lengths for each layer greedily. We proceed recursively; storing the current points along the perimeter and iterating along them, finding the best possible position for each gate and updating the perimeter points accordingly.
- For finding this optimal position, we have defined Pin class, Gate class and Network class. The network class stores the maximum and minimum x and y coordinates of the structure created till now i.e. the span of all the gates we've placed till now. This span defines the bounding box for the set of current pin positions. After placing a gate at any position, we update these parameters and compute the change in semi-perimeter of the bounding box. The position along the parameter, giving minimum change is chosen as the optimal one.

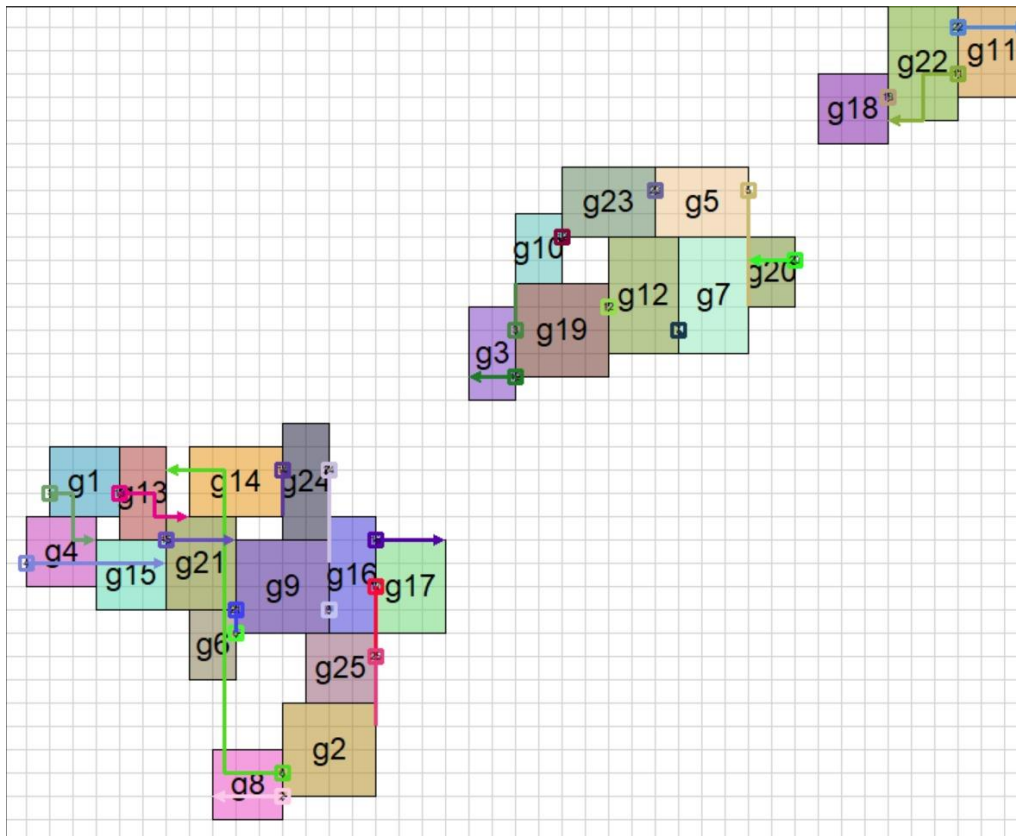


Figure 1: greedy algo

Time complexity analysis:

- No. of gates (N) ≤ 1000 ; Max width ≤ 100 ; No. of pins (P) ≤ 40000
- The overall process involves two major phases: sorting the gates and performing a breadth first search on the gates:
 - Time complexity for sorting the gates: $O(N \cdot \log(N))$
 - Time complexity for the BFS process: $O(N)$
- Now, considering the operations within each cluster; let n and p denote the no. of gates and pins respectively in a cluster. Within each cluster, the following operations are performed:
 - Assigning a network to each pin: $O(p)$
 - Iterating through each gate in the cluster: $O(n)$
- Within each cluster, the following operations are performed:
 - Iterating through the entire current perimeter: $O(n \cdot 400)$
 - Insider each perimeter iteration, we iterate through the gate perimeter: $O(400)$
- Hence, the time complexity per cluster: $O(p + 160000 \cdot n^2)$
- Let C be the no. of clusters. This results in an overall time complexity of : $O(N \cdot \log(N) + N + C(p + n^2 \cdot 160000))$
- Since $C \cdot p = P$ and $C \cdot n = N$, we can simplify this expression to: $O(N \cdot \log(N) + N + P + 160000 \cdot N \cdot n)$, where n is the average no of gates per cluster

- Worst Case Estimation: In the worst case, the total number of operations is thereby approximated to 16×10^{10} . In practice, the process took 5-10 minutes to complete for the worst-case scenario.

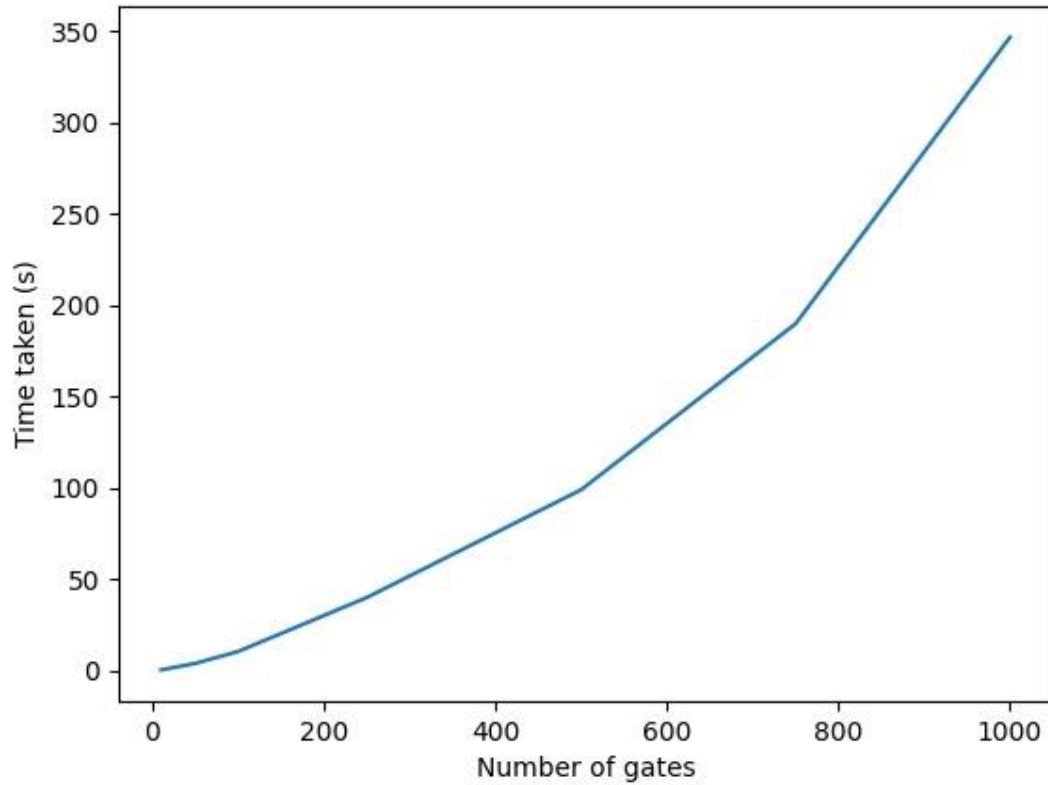


Figure 2: N vs t graph is approximately a parabola which verifies our hypothesis

No. of gates (N) in the cluster	Time taken (t)
10	0.33s
50	3.9s
100	10.44s
250	40.1s
500	99.07s
750	189.97s
1000	347.01s

Test Cases:

- For cases where number of pins might exceed 40000, we have modified the randint range accordingly to give an expected number of total pins around this value
- We initially modelled the left side of a gate having only one connection but later extended this to the left side having no such constraint as practical circuits might have multiple inputs as well as multiple outputs
- A local visualizer has been used to highlight the wirings clearly

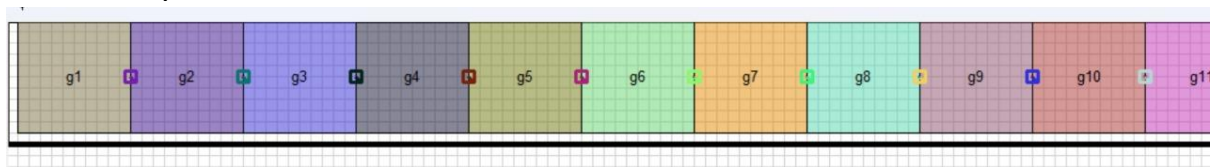
1) *Random 1000*

- Generates 1000 gates using randint function (dimension range 1 to 100) and having a random number of pins and connections among each other
- Tests the ability of the algorithms to work on random circuits with no well-defined pattern of gate sizes and interconnections and also the average case time complexity of the algorithm under the given constraints (3-10 minutes for different values)

(visualization not attached due to large number of gates)

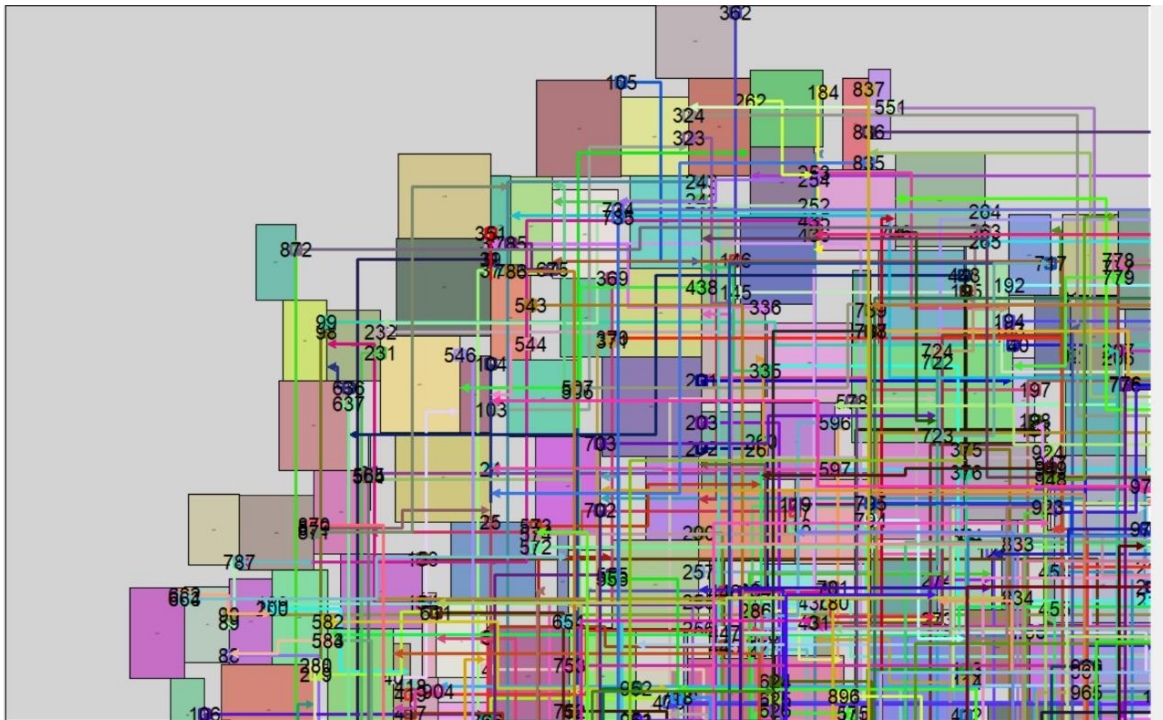
2) *Pairwise connected gates*

- 50 gates of equal dimensions with only single input pin and single output pin, identically placed.
- Tests if the algorithm works for circuits consisting NOT gates and Buffer gates having single I/O. The output wire length matches the expected answer, 0.



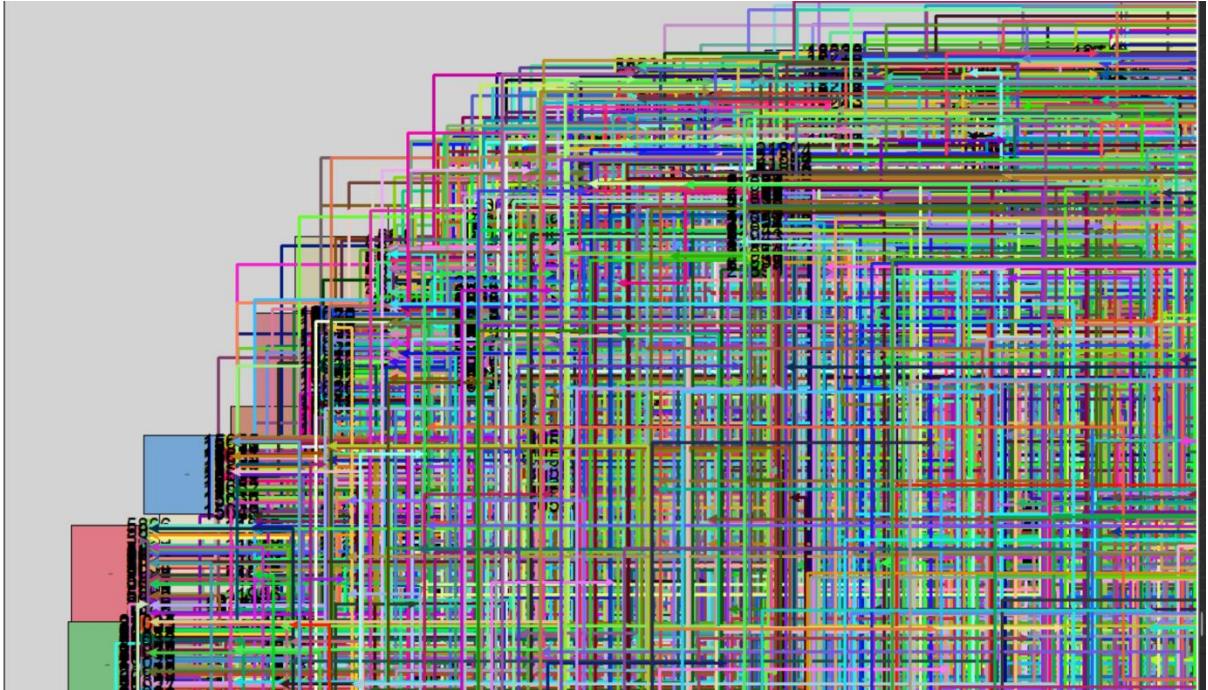
3) *Sparsely connected gates*

- Samples 500 gates of varying dimensions and randomly assigned pin numbers and defines connections such that each gate is connected to at most 3 other gates
- Modelling circuits where the interconnections between gates are sparse.



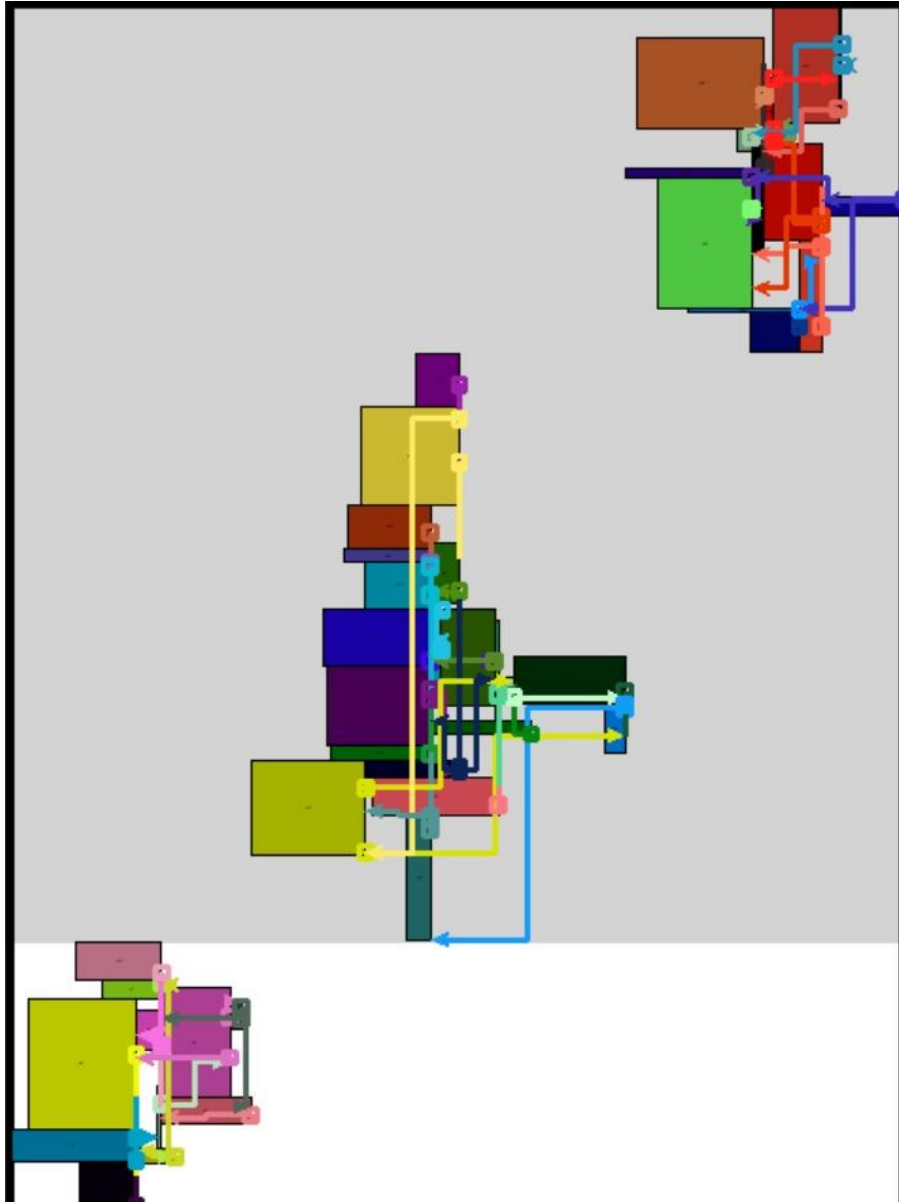
4) *Densely Connected Gates*

- Samples 500 gates of varying dimensions and randomly assigned pin numbers and defines connections such that each gate is connected to a random number of gates between 30 and 60
- Modelling circuits where the interconnections between gates are dense



5) Clusters

- Divides 45 gates into 3 clusters of random sizes and generates pins and connections for each cluster
- Modelling circuits having disconnected components of various sizes and the ability of the code to process these independently



Area for improvement:

- In certain cases (included in submission named "suboptimal.txt"), involving complex, cyclic connections and multiple wires connecting the same gates, our algorithm exhibited suboptimal performance. The primary reason is that the current algorithm does not represent the circuit as a "weighted graph." By modelling gates as nodes in a weighted graph, where the number of connections between gates is assigned as edge weights, the algorithm's accuracy could be slightly improved.